

Donghyung Lee

DevOps Engineer | Resume & Career Description

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

Introduction

Hello, I am Donghyung Lee, a DevOps Engineer.

Targeting 99.9% infrastructure availability, I have continuously improved operational efficiency by codifying every change as IaC and turning incident response into postmortem-driven assets. My greatest strength is going beyond routine operations to interrogate the meaning and purpose of each task and pursue a better direction.

I design and build AWS/GCP cloud and OpenStack on-premises PaaS infrastructure, operate Kubernetes (EKS, GKE, on-premises), apply Terraform and Ansible IaC, and build CI/CD pipelines on GitLab CI/CD, GitHub Actions, ArgoCD, and Jenkins. I've built monitoring/logging systems with PLG and ELK stacks, and handle operations automation directly in Python and Bash.

I've also been expanding my strengths into data engineering and AI/ML infrastructure — building data pipeline automation by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler (multi-source → BigQuery → Google Sheets, Terraform IaC) and deploying an internal LLM service (Ollama + Open WebUI) on GPU servers. Alongside that, I develop in-house automation / DevOps tools myself such as git-bridge, a Slack APK deploy bot, and static-file-server.

Across 3 companies, I have led AWS → GCP migration, hybrid architecture design, and on-premises PaaS deployment as the sole or primary owner, achieving 20% AWS cost reduction via hybrid transition, 50% reduction via GCP migration optimization, 50-67% deployment time cuts, and 90%+ automation of manual work.

I aim to continue contributing to infrastructure architecture advancement and organizational productivity improvement.

Strengths

Quick Understanding & Adaptation

- Quickly adapt to new environments and tools, contributing immediately to diverse projects

Systematic Documentation & Knowledge Sharing

- Document all work processes and share with team, contributing to organizational knowledge assets

Record-Based Continuous Improvement

- Manage all tasks including incident response, infra changes, and troubleshooting through records to prevent recurrence and drive improvement

Full-Stack Infrastructure Experience

- End-to-end infrastructure experience spanning cloud (AWS, GCP), on-premises, networking, CI/CD, and monitoring

Skills

Cloud	AWS, GCP, OpenStack
Container Orchestration	Kubernetes (EKS/GKE Multi-cluster Ops, On-premises HA with CNI/CSI Optimization)
Infrastructure as Code	Terraform, Ansible, Vagrant, Packer
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, Jenkins
Monitoring & Logging	PLG Stack, ELK Stack, EFK Stack, Thanos
Automation	Go, Python, Bash Scripting
Database	MySQL, PostgreSQL, Redis, MongoDB
OS & Servers	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- IaC-based infrastructure automation and version control using Terraform (100% GCP resource codification)
- On-premises to cloud hybrid architecture design and migration experience

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design and enhancement using GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction with 3x weekly deployments shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Jenkins-based Unity Android/iOS build automation and Slack-integrated deployment notifications

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Integrated metric/log monitoring with PLG Stack (Prometheus, Loki, Grafana)
- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- 60-70% MTTR improvement, SLI/SLO-based 99.9% service availability achieved

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- 90%+ manual work reduction through data collection, deployment, and document management automation
- Internal LLM service (Ollama + Open WebUI) ensuring 100% internal security guideline compliance and dev team productivity improvement
- Internal tool development: GitLab Webhook doc sync, Slack Bot APK deployment notifications

Kubernetes Platform Modernization & Open-Source Contribution

Zero-downtime migration of network/logging stacks and releasing reusable public charts as open source

- Migrated 11 ingress-nginx instances to a single NGINX Gateway Fabric control-plane + 11 Gateway CRs with zero downtime (11/11 cutover, wildcard TLS consolidated)
- Migrated the ECK Operator-based logging stack from a local chart to an OCI chart with zero downtime (CR/PVC names preserved, cluster_uuid unchanged, no data loss)
- Released 4 open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck, certmanager-letsencrypt) and multiple composite GitHub Actions on ArtifactHub / GitHub Marketplace

Data Engineering & AI/ML Infrastructure

End-to-end ownership from GCP data pipelines to in-house LLM services

- Built a multi-source (MongoDB · CloudSQL · GA · Dune) → BigQuery → Google Sheets pipeline by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler — entire infrastructure 100% Terraform IaC
- Deployed an internal LLM service (Ollama + Open WebUI) on a Kubernetes-hosted on-premises GPU server (RTX 3060), saving ~\$100/month by replacing external AI SaaS subscriptions and ensuring 100% internal security compliance
- 95% data-collection automation and 0% data gap rate by eliminating human error

Professional Experience (Detailed)

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As a dedicated DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture.

Currently providing secondary technical support and operations for the live game 'Souls' in collaboration with the publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of \$60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud elasticity unnecessary.

Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.

Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version. Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- Intermittent timeouts during ClusterIP service communication in Kubernetes IPVS mode. Resolved through Cilium eBPF packet flow analysis and IPVS conntrack threshold tuning, and built a Cilium Network Policy-based zero-trust security model to enhance inter-pod traffic visibility and security
- Manual image sync between Harbor (on-prem) and ECR (AWS) was driving frequent cross-cluster push/pull through the public internet, pushing up build time, cost, and failure rate. Automated bidirectional sync — Harbor's native event-based Replication for Harbor → ECR (auto-replicate on every push), and an EventBridge-triggered GitHub Actions workflow running `skopeo copy` for ECR → Harbor — cutting deployment time by 40%
- During rolling deploys, the ALB target deregistration delay (lowered from the default 300s to 60s) didn't line up with Pod SIGTERM handling, causing some game-session connection drops. Added a 60s sleep in the preStop hook (failing the readiness probe so ALB marks the target unhealthy and stops sending new requests), set terminationGracePeriodSeconds to 90s (ALB drain 60s + in-flight handling 30s), and implemented a SIGTERM handler in the game-server container — achieving zero-downtime deploys with 0 connection drops

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% (\$60,000 → \$48,000), reinvesting ~\$144,000 annual savings into new project infrastructure for resource reinvestment cycle
- Offloaded dev servers to on-premises, cutting ~\$1,000/month in AWS resource costs and optimizing resource usage
- Ensured operational stability and prevented failure propagation through environment role separation

Tech Stack

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. Eliminated Docker-in-Docker dependency using Kaniko for Docker image builds, and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- ArgoCD Application's syncPolicy.automated.selfHeal=true reverted operators' temporary hot-fixes instantly, making incident debugging hard. Split policy by environment (prod: selfHeal=false + manual sync windows; dev/staging: selfHeal=true) and integrated Slack alerts for every sync event to gain both change traceability and stability
- Package installation failure on CI Runner due to GPG key expiration after GitLab upgrade. Automated GPG key renewal process to prevent recurrence
- Build time spikes due to cache invalidation during Kaniko builds. Resolved with -cache=true --cache-ttl=24h --snapshot-mode=redo option combination

Results

- 67% deployment time reduction (30min → 10min), 90% manual work automation
- Deployment automation increased weekly deployments 3x+ (1-2 → 5-7), shortening Time to Market for new features, while deployment resource efficiency enabled the dev team to focus on core feature development

Tech Stack

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

Project 3: Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement)

Contribution 100% (System design & implementation) 2025.06 ~ Present (Operations & Enhancement)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents.

Troubleshooting

- Logstash parsing failures due to null bytes in game server logs. Resolved by pre-removing with mutate filter gsub
- Elasticsearch field mapping conflicts. Resolved with explicit index template mappings and type conversion filters
- Index read-only due to disk watermark exceeded. Stabilized with ILM policy auto-deleting indices older than 7 days
- [EFK] Fluent Bit NFS WAL write failure. Resolved with custom PVC template patch + upgrade script auto-preserve
- [EFK] Fluentd buffer overflow log loss. Resolved by tuning chunk_limit_size/flush_interval
- [ECK] Kibana 9 secure-cookie enforcement blocked HTTP login. Worked around with tls.disabled + publicBaseUrl http:// + secureCookies:false
- [ECK] 3.3.x stackconfigpolicy-controller cache-scope mismatch caused reconcile errors every 17 minutes. Worked around with managedNamespaces:[] (cluster-wide watch); RBAC was already cluster-scoped
- [ECK] No Fluentd ES9 variant → ensured ES 9.0 compatibility with ES8 variant image via fluent-plugin-elasticsearch 5.4.4 and suppress_type_name:true, verified end-to-end via active marker

Approach

Phase 1 (ELK Setup): Built a centralized logging system with Elasticsearch + Logstash + Kibana + Filebeat. Filebeat collected node/container logs, Logstash handled JSON parsing and field mapping, Elasticsearch stored them, and Kibana dashboards provided real-time monitoring and search.

Phase 2 (EFK Migration): Migrated to a Fluent Bit + Fluentd EFK Stack to relieve Logstash's JVM memory burden (1GB+) and Filebeat's Helm-ecosystem mismatch. Lightweight C-based Fluent Bit ran as a DaemonSet with automatic Kubernetes metadata tagging, while Fluentd handled multi-output processing/filtering before forwarding to Elasticsearch. A chart-upgrade automation script preserved custom PVC templates across upstream chart auto-upgrades.

Phase 3 (ECK Operator + Stack 9.0): With Elastic's Helm Chart updates halted and CRD-based declarative management becoming mainstream, performed an 8.5.1 → 9.0.0 major upgrade and deployed ECK Operator 3.3.2 in the elastic-system namespace. Redefined ES/Kibana as Custom Resources (CR wrapper local chart), executed a zero-downtime cutover after parallel monitoring → logging ns deployment. Established automatic TLS rotation (1y / 30d), Helm-templated elastic account password, and rolling upgrades with a single-line CR spec.version change.

Results

- 90% log search time reduction, 70% MTTR improvement maintaining 99.9% availability
- 70% memory reduction via EFK migration (Logstash 1GB → Fluent Bit 50MB)
- 1GB daily log real-time processing, unified Helm management with automated upgrades
- ECK migration removed manual ops burden for TLS, passwords, StatefulSet upgrades; Stack 9.0 major upgrade closed a 2-year update gap
- Operator secureMode + ServiceMonitor auto-registers controller-runtime metrics into kube-prometheus-stack

Tech Stack

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% 2025.02 ~ Present

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Auto-notification to Slack with QR code on Jenkins build completion. 90% QA team delivery time reduction

Tech Stack

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.08)

Auto-sync on push via GitLab Webhook + Google Drive API. 80% document management time reduction

Tech Stack

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. Internal LLM Service Deployment (4 weeks, 2026.01)

Deployed Ollama + Open WebUI on Kubernetes on on-premises GPU server (RTX 3060). Serves 15 developers with avg 100 queries/day, saving ~\$100/month by replacing external AI SaaS subscriptions.

100% internal security guideline compliance and eliminated data leakage risks

Tech Stack

Ollama, Open WebUI, Kubernetes, NFS, GPU (RTX 3060)

4-4. Git Repository Mirroring Tool Development (4 weeks, 2026.02)

Developed Go-based bidirectional Git mirroring tool (git-bridge). Ensured code quality with unit tests and GitHub Actions CI/CD pipeline.

Auto-sync across CodeCommit/GitLab/GitHub for 10 repos with avg 30 sync events/day, 100% automation

Tech Stack

Go, AWS SQS, GitLab Webhook, GitHub Webhook, Kubernetes, Docker

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Developed a lightweight Go-based static file server (static-file-server) in-house and operated a Helm repository on GitHub Pages. Features: dark-mode directory listing UI, file preview (image/video/PDF/text), search with URL-hash sharing, multi-select + ZIP batch download, Gzip, Prometheus metrics, JSON logging, automatic APK Content-Type.

Deployed to Kubernetes via an in-house Helm chart (NFS nfs-client-nopath, Retain, 5Gi), fully replacing the legacy halverneus/static-file-server:v1.8.11 (moved to _deprecated). Handles ~30 downloads/day and 5 APK builds/week (daily build). Released as open source under Apache-2.0.

Tech Stack

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% 2025.12 ~ Present

To improve cluster visibility, operational stability, and network modernization, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API. Released both the CR management structure derived from the NGF migration and the ECK Operator-based logging stack as separate open-source Helm charts.

5-1. Monitoring & Alerting Enhancement (2026.01 ~ Present)

Designed 18 custom alert rules (9 Node, 2 Pod, 4 Cilium) and 10 custom Grafana dashboards based on kube-prometheus-stack and integrated MySQL/Redis/Elasticsearch/PostgreSQL exporters. Deployed node-exporter on 4 physical servers and VMs via Ansible, and collected Cilium CNI agent/operator metrics via kubernetes_sd_configs.

Configured severity-based Slack routing with inhibit rules in Alertmanager to improve alert quality.

Tech Stack

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ 2026.04)

Kubespary upgrade automation script for pre-flight, backup, compatibility checks + 9-step health check script reducing verification time by 90%.

Upgrade framework across 18 Helm charts + 4 canonical templates synchronizer (check for CI drift detection, apply for batch propagation) maintaining script body consistency.

Automated Kubernetes certificate renewal script + crontab (10 nodes, every 6 months at 3 AM) proactively preventing API Server outages from certificate expiration

Tech Stack

Kubespary, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Migrated 11 ingress-nginx controller instances (default + public-a~j with 11 pinned MetalLB LoadBalancer IPs) to a single NGF 2.x control-plane + 11 Gateway CRs, preserving MetalLB IPs so DNS/firewall stayed unchanged via parallel deployment and gradual cutover (30-60s actual downtime per class).

Executed Phase 0~7+ (MetalLB pool expansion → NGF install → 5 observability apps → HTTP-only / 9 ApplicationSet apps → HTTPS-enforced

apps (Harbor, Vaultwarden) → IP-swap cutover 11/11 → Ingress cleanup). Mapped 10 ingress-nginx annotations 1:1 to Gateway API + NGF CRDs (HTTPRoute filters, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy) and unified the HTTPRoute values schema across four charts for batch migration of 9 ApplicationSet apps. Consolidated two per-app self-signed TLS Secrets into a single wildcard-concrit-tls (10-year validity), cutting manual renewal overhead by 50%.

Tech Stack

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. nginx-gateway-cr Open-Source Helm Chart Release (2026.04)

Refined the CR structure extracted during the NGF migration and released an open-source Helm chart that deploys the tenant-level resources (Gateway, NginxProxy, ReferenceGrant, ServiceMonitor, PodMonitor) that the NGF upstream chart does not create. Includes a multi-Gateway-friendly schema and PodMonitor templates recommended for NGF 2.x, distributed via both an OCI registry and a traditional Helm repo (Apache-2.0, ArtifactHub)

Tech Stack

Helm Chart, Gateway API, NGINX Gateway Fabric, Prometheus Operator, OCI Registry, ArtifactHub

5-5. ECK Operator-based Elasticsearch / Kibana Helm Chart Release (2026.04)

Designed charts/elasticsearch-eck + charts/kibana-eck to deploy ECK Operator CRs, Secrets, HTTPRoute, Ingress, ServiceMonitor, and NetworkPolicy from a single set of values, with values.schema.json (draft-07) validation plus Minimal/HA presets and per-environment storageClass mapping in the README. Published simultaneously across three channels — OCI (ghcr.io/somaz94/charts), a gh-pages Helm repo, and ArtifactHub monitoring-logging category (Apache-2.0, 0.1.1).

Zero-downtime cutover on the mgmt cluster preserved every CR / PVC / Secret name — cluster_uuid stayed unchanged, with zero Elasticsearch pod restarts and a single Kibana rolling restart, i.e. no data loss. HA rolling scenarios passed 3/3 on a local kind cluster; end-to-end install on the real cluster reached Ready in 71s for Elasticsearch and 57s for Kibana. Also added a new canonical scripts/upgrade-sync/templates/oci-chart-version.sh template to tighten the internal OCI chart-version tracking procedure.

Tech Stack

Helm Chart, ECK Operator, Elasticsearch, Kibana, Gateway API, OCI Registry, ArtifactHub, Prometheus Operator, Kubernetes

Project 6: Infrastructure Security System

Contribution 100% 2026.04 ~ Present

6-1. Vaultwarden Password Management System (2026.04)

Deployed Vaultwarden on K8s via Helm, GitLab SSO integration, org/collection access control. Automated daily backup (30-day retention) + interactive restore.

Migrated 70 users and 50 secrets into centralized management

Tech Stack

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game and blockchain startup, leading the large-scale cloud migration project from AWS to GCP.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Reconstruction

Contribution: Infra design/migration lead 70%, CI/CD pipeline 100% 7 months (2023.05 ~ 2023.12)

Problem

Continuously rising AWS costs (\$10,000+/month, high NAT Gateway and data transfer costs), need for GCP global load balancing for multi-region game services, and the legacy Jenkins-based deployment system's configuration complexity and lack of deployment history tracking

Approach

Adopted Shared VPC (Host / Service Project separation) and Workload Identity Federation so GitHub Actions could authenticate to GCP via short-lived tokens without service-account keys. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for

pre-validation, then performed the final NS swap during a single scheduled cutover for zero-downtime migration. Worked around Filestore's 2.5TB SSD minimum cost by self-hosting an NFS server on Compute Engine (pd-balanced 1TB). Configured Cloud Armor with region blocking + IP allow-list WAF policy, and split the Cloud CDN URL Map into HTTP / HTTPS variants to enforce HTTP → HTTPS redirect alongside static-asset caching.

Troubleshooting

- Routing issues during AWS ALB → GCP Global LB transition. Reconfigured after analyzing GCP NEG structure
- Data corruption during RDS → Cloud SQL migration due to character set differences. Resolved with pre-validation scripts and step-by-step integrity checks
- Network design issues due to AWS/GCP subnet model differences. Redesigned CIDR and converted to GCP tag-based firewalls
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response. Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig
- Game client file downloads failed after the CDN migration due to missing HTTP → HTTPS redirect. Resolved by splitting the URL Map into HTTP / HTTPS variants (default_url_redirect.https_redirect=true)

Results

- 50% cloud cost reduction (\$10,000 → \$5,000/month), 50% deployment time reduction, 95% rollback reduction
- Full GCP infrastructure Terraform IaC (TFLint/Checkov static analysis, Terratest unit testing), 100% deployment history tracking via Git commits
- Minimized downtime with resource-copy migration; final DNS cutover executed during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions

Tech Stack

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

Project 2: Game Data Collection Automation System

Contribution 100% 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily, with frequent data loss caused by human error

Approach

Loaded multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery — MongoDB via Dataflow ETL, CloudSQL via direct connection, GA/Dune via API — then triggered Daily / Monthly via Cloud Functions + Cloud Scheduler and pushed analyst-facing sheets via the Google Sheets API. A separate Cloud Function periodically deduplicated BigQuery data.

The entire infrastructure (datasets, Cloud Functions, schedulers, storage, Artifact Registry) was managed as Terraform IaC.

Results

- 95% data collection automation and 0% data gap rate by eliminating human error

Tech Stack

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Docker

Project 3: PLG Stack Monitoring System

Contribution 100% 1 month (2024.04)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive responses with excessive time spent on root-cause analysis

Results

- SLI/SLO establishment and Alertmanager alarm noise optimization, 80% proactive detection, 60% response time reduction, availability 99.5% → 99.9%

Tech Stack

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support, designing on-premises cloud infrastructure for financial and public-sector clients and conducting operations training.

Key Projects: PaaS Solution · DP Training · Certificate Automation · Ansible Configuration Management

Results

- PaaS solution for 5 clients, Kubernetes HA 99.9% availability
- Established DP's autonomous Kubernetes operations (6-month training, avg 10 attendees · 4 hrs per session)
- 10x certificate renewal extension (1yr → 10yr), 50hrs annual ops savings
- Ansible-based server provisioning and configuration management automation eliminating configuration drift issues

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, kubeadm, Rocky Linux/CentOS

ETech System

2022.01 ~ 2022.04

System Engineer

Hardware Server and SAN storage construction. Transitioned to DevOps field for career growth.

Opensource Projects

Kubernetes CRD

[k8s-namespace-sync](#) — Automatically sync Kubernetes namespace resources across clusters

[helios-lb](#) — Custom load balancer controller for Kubernetes

[network-policy-generator](#) — Kubernetes NetworkPolicy auto-generation controller

GitHub Actions

[Compress Decompress Action](#) — Compress/decompress files in workflows

[Image Tag Updater](#) — Auto-update image tags in YAML/Helm files

[Go Git Commit Action](#) — Go-based Git commit creation and push

[Extract Commit Action](#) — Extract Git commit SHA, message, and author information

[Multi Git Mirror Action](#) — Automate mirroring across multiple Git repositories

[Kube Diff Action](#) — Compare and report Kubernetes resource changes

Ansible Galaxy

[Ansible K8s IAC Tool](#) — Automated K8s and IaC tool installation collection

[Ansible User Management](#) — Linux user and SSH key management role

DevOps Tools

[bash-pilot](#) — AI-powered Bash command assistant CLI tool

[git-bridge](#) — Git repository mirroring and sync CLI tool

[static-file-server](#) — Go-based static file server with directory UI, file preview, search, ZIP batch download

Chrome Extensions

[Dev Toolkit](#) — Developer utility collection

[QRify](#) — Instant QR code generation for URLs/text

Education

Chungnam National University - Public Administration (Transfer)

2018.03 ~ 2020.08

National Institute for Lifelong Education - Business Administration (Academic Credit Bank)

2016.09 ~ 2018.02

Certifications

Engineer Information Processing	2021.11
AWS Certified Solution Architect - Associate (Expired)	2021.11
Network Administrator Level 2	2021.10
Linux Master Level 2	2021.10
Computer Specialist Level 1	2021.04
TOEIC Speaking Test - 130/Intermediate Mid 3 (Expired)	2021.02

Study

T101(Terraform) Study	2023.08 ~ 2023.10
AEWS(EKS) Study	2023.04 ~ 2023.06
PKOS(Kubernetes) Study	2023.01 ~ 2023.02
Cloud Security Training	2022.02 ~ 2022.04
Security Solution Operation Specialist Training Program	2021.07 ~ 2021.12